PRACT 3 BANK

October 31, 2026

[1]:
```
# 1. Import required libraries

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

[2]:
```
# 2. Load the dataset

df = pd.read_csv(r"E:\LP3_LAB Manual\ML PRAC\PRACT3\Churn_Modelling -␣
↪Churn_Modelling.csv")

print("Dataset loaded successfully")
print(df.head())
```

```
Dataset loaded successfully
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female   42
1          2    15647311      Hill          608     Spain  Female   41
2          3    15619304      Onio          502    France  Female   42
3          4    15701354      Boni          699    France  Female   39
4          5    15737888  Mitchell          850     Spain  Female   43

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0       2       0.00              1          1               1
1       1   83807.86              1          0               1
2       8  159660.80              3          1               0
3       1       0.00              2          0               0
4       2  125510.82              1          1               1

   EstimatedSalary  Exited
```

```
0        101348.88       1
1        112542.58       0
2        113931.57       1
3         93826.63       0
4         79084.10       0
```

[3]:
```
# 3. Remove unnecessary columns

df = df.drop(
    columns=["RowNumber", "CustomerId", "Surname"]
)
```

[4]:
```
# 4. Convert categorical columns into numerical values

df = pd.get_dummies(
    df,
    columns=["Geography", "Gender"],
    drop_first=True,
    dtype=int
)
```

[5]:
```
# 5. Separate input features and target variable

X = df.drop("Exited", axis=1)
Y = df["Exited"]

print("\nInput features:")
print(X.head())

print("\nTarget variable:")
print(Y.head())
```

```
Input features:
   CreditScore  Age  Tenure    Balance  NumOfProducts  HasCrCard  \
0          619   42       2       0.00              1          1
1          608   41       1   83807.86              1          0
2          502   42       8  159660.80              3          1
3          699   39       1       0.00              2          0
4          850   43       2  125510.82              1          1

   IsActiveMember  EstimatedSalary  Geography_Germany  Geography_Spain  \
0               1        101348.88                  0                0
1               1        112542.58                  0                1
2               0        113931.57                  0                0
3               0         93826.63                  0                0
4               1         79084.10                  0                1
```

```
   Gender_Male
0            0
1            0
2            0
3            0
4            0

Target variable:
0    1
1    0
2    1
3    0
4    0
Name: Exited, dtype: int64
```

[6]:
```python
# 6. Split the dataset into training and testing sets

X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.20,
    random_state=42,
    stratify=Y
)

print("\nTraining data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (8000, 11)
Testing data shape: (2000, 11)
```

[7]:
```python
# 7. Feature scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[8]:
```python
# 8. Create the neural network classifier

model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation="relu",
    solver="adam",
    max_iter=500,
    random_state=42
)
```

[9]:
```
# 9. Train the neural network

model.fit(X_train, Y_train)

print("\nNeural Network Model trained successfully")
```

```
Neural Network Model trained successfully
```

[10]:
```
# 10. Predict customer churn

Y_pred = model.predict(X_test)
```

[11]:
```
# 11. Evaluate the model

accuracy = accuracy_score(Y_test, Y_pred)

print("\nModel Accuracy:", round(accuracy * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(Y_test, Y_pred))

print("\nClassification Report:")
print(classification_report(
    Y_test,
    Y_pred,
    target_names=["Not Exited", "Exited"]
))
```

```
Model Accuracy: 87.25 %

Confusion Matrix:
[[1551   42]
 [ 213  194]]

Classification Report:
              precision    recall  f1-score   support

  Not Exited       0.88      0.97      0.92      1593
      Exited       0.82      0.48      0.60       407

    accuracy                           0.87      2000
   macro avg       0.85      0.73      0.76      2000
weighted avg       0.87      0.87      0.86      2000
```

[12]:
```
# 12. Predict churn for a new customer

new_customer = pd.DataFrame([{
    "CreditScore": 650,
    "Age": 40,
    "Tenure": 5,
    "Balance": 80000,
    "NumOfProducts": 2,
    "HasCrCard": 1,
    "IsActiveMember": 1,
    "EstimatedSalary": 75000,
    "Geography_Germany": 0,
    "Geography_Spain": 0,
    "Gender_Male": 1
}])
```

[13]:
```
# Ensure the new customer's features match training columns

new_customer = new_customer.reindex(
    columns=X.columns,
    fill_value=0
)

# Scale the new customer's data

new_customer_scaled = scaler.transform(new_customer)

# Predict the result

prediction = model.predict(new_customer_scaled)
```

[14]:
```
# Display the result

if prediction[0] == 1:
    print("\nPrediction: Customer is likely to leave the bank.")
else:
    print("\nPrediction: Customer is likely to stay with the bank.")
```

```
Prediction: Customer is likely to stay with the bank.
```

[15]:
```
# Display churn probability

probability = model.predict_proba(new_customer_scaled)[0][1]

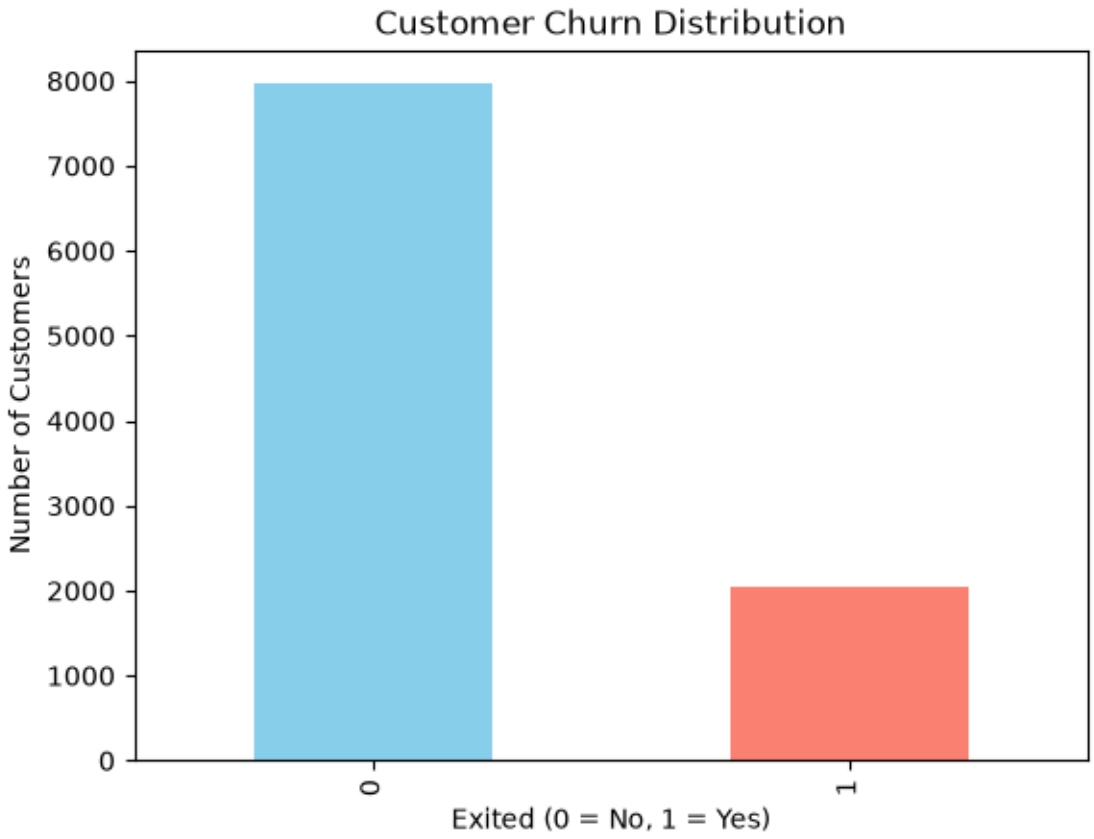
print("Probability of customer leaving:", round(probability * 100, 2), "%")
```

```
Probability of customer leaving: 1.08 %
```

[18]:
```
# Shows how many customers stayed and how many left the bank.
import matplotlib.pyplot as plt

df["Exited"].value_counts().plot(
    kind="bar",
    color=["skyblue", "salmon"]
)

plt.title("Customer Churn Distribution")
plt.xlabel("Exited (0 = No, 1 = Yes)")
plt.ylabel("Number of Customers")
plt.show()
```



[ ]: